Reviewer Pack — Minimal Order of Category Monoids and Frege Proof Depth Ineq…

Entry 4792defc9e57 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 19:28:43 UTC

Minimal Order of Category Monoids and Frege Proof Depth Inequality

Entry ID: `4792defc9e57` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 19:28:43 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Monoidal Categories) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

> For every Boolean formula φ, the minimal order of a monoid in the category of endofunctors of the poset [φ], denoted as min_order(φ), is upper-bounded by the Frege proof depth of φ, i.e., min_order(φ) ≤ f_depth(φ), where f_depth(φ) = Θ(n^c) for some constant c.

Rationale (proposer's reasoning):

> Monoidal categories have been used to encode computational structures, and their monoids can capture essential properties of those structures. By examining the minimal order of category monoids associated with Boolean formulas, we might uncover a new perspective on Frege proof complexity that could potentially lead to improved complexity lower bounds.

Taxonomy category: `Category Theory × Boolean Satisfiability` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `dddda1b686c4c626`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all Boolean formulas φ with n variables, the ratio of min_order(φ) to f_depth(φ) is less than or equal to a constant c, and falsified if any formula φ has a ratio greater than c.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (1): - `category theory AND Frege proof complexity AND monoid order`

Top relevant hits considered: - [http://arxiv.org/abs/2503.17274v3] Composable Uncertainty in Symmetric Monoidal Categories for Design Problems (Extended Version) - [http://arxiv.org/abs/1912.10642v7] Notes on Category Theory with examples from basic mathematics - [http://arxiv.org/abs/1802.09555v2] Involutive categories, colored $*$-operads and quantum field theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def poset_size(formula):
        if formula == 'true' or formula == 'false':
            return 1
        elif formula.startswith('(') and formula.endswith(')'):
            left, op, right = formula[1:-1].split()
            return max(poset_size(left), poset_size(right)) + 1
        else:
            return 2

    def frege_proof_depth(formula):
        if formula == 'true' or formula == 'false':
            return 0
        elif formula.startswith('(') and formula.endswith(')'):
            left, op, right = formula[1:-1].split()
            return max(frege_proof_depth(left), frege_proof_depth(right)) + 1
        else:
            return 2

    def min_order(formula):
        if formula == 'true' or formula == 'false':
            return 1
        elif formula.startswith('(') and formula.endswith(')'):
            left, op, right = formula[1:-1].split()
            return max(min_order(left), min_order(right)) + 1
        else:
```

```
            return 2

    def generate_formula(n):
        if n == 0:
            return random.choice(['true', 'false'])
        elif n == 1:
            return f'({generate_formula(0)} {random.choice(["&", "|"])} {generate_formula(0)})'
        else:
            return f'({generate_formula(n-1)} {random.choice(["&", "|"])} {generate_formula(n-1)})'

    instances_tested = 0
    n_max = 0
    total_metric_value = 0

    for n in [5, 10, 15, 20, 30, 40]:
        if n > 40:
            break

        for _ in range(5):
            formula = generate_formula(n)
            instances_tested += 1
            n_max = max(n_max, n)

            poset = poset_size(formula)
            f_depth = frege_proof_depth(formula)
            min_order_val = min_order(formula)

            if f_depth == 0:
                continue

            ratio = Fraction(min_order_val, f_depth)
            total_metric_value += ratio

    mean_metric_value = total_metric_value / instances_tested
    conjecture_holds = all(Fraction(min_order(formula), frege_proof_depth(formula)) <= Fraction(1,

    return {
        "metric_name": "Min Order to Frege Depth Ratio",
        "metric_value": float(mean_metric_value),
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
```

```
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54dd729d.py", line 97, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54dd729d.py", line 69, in run_trial
    poset = poset_size(formula)
            ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54dd729d.py", line 25, in poset_size
    left, op, right = formula[1:-1].split()
    ^^^^^^^^^^^^^^^
ValueError: too many values to unpack (expected 3)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot determine if the conjecture is supported or falsified based on the provided results. | next: Investigate and fix the crash in the test code to continue testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14240
2	preregistration	ollama_remote	glm4:latest	0	0	10179
3	novelty	ollama_remote	glm4:latest	0	0	11472
4	novelty	ollama_remote	glm4:latest	0	0	9765

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16203
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15311
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9122
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11610
9	judge	ollama_remote	glm4:latest	0	0	20201

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 118104 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4792defc9e57.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4792defc9e57.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4792defc9e57.tar.gz` (if generated)